

Advanced Natural Language Processing: AdvNLP

Lesson 11: Retrieval Augmented Generation

Table of Contents

1. Overcoming the limitations of LLMs with RAG
2. Chunking and indexing texts
3. Similarity search in vector databases
4. Optimization and evaluation of RAG
5. Conclusions

1. OVERCOMING THE LIMITATIONS OF LLMS WITH RETRIEVAL AUGMENTED GENERATION

Recap – The nature of LLMs

LLMs

- leverage pre-trained deep neural networks trained on vast amounts of openly available scraped data to **predict next words**,
- generate probable content by **self-supervised pre-training** based on internalized knowledge in an **auto-regressive, non-deterministic** way,
- extend training by **fine-tuning** on carefully compiled data to **align with user preferences**,
- apply **decoding strategies** to generate fluent textual content

$$P(w_1, w_2, \dots, w_n) = \prod_{k=1}^n P(w_k | w_1, w_2, \dots, w_{k-1})$$

Pre-training: Maximize the likelihood of the entire sequence of tokens

Recap – LLM hallucination types

Factuality hallucination:

- **Factuality contradiction:** LLM's output contains facts that can be grounded in real-world information, but contradict this information
- **Factuality fabrication:** LLM's output contains facts that are unverifiable against established real-world knowledge

Faithfulness hallucination:

- **Instruction inconsistency:** LLM's output deviate from a user's directive.
- **Context inconsistency:** LLM's output is incompatible with the provided contextual information
- **Logical inconsistency:** LLM output exhibits internal logical contradictions

Category	Type	Example Dialog
Factuality Hallucination	Factual Contradiction	Instruction: What are Thomas Edison's main contributions to science and technology? Response: Thomas Edison developed the first practical telephone and invented the light bulb.
	Factual Fabrication	Instruction: What are the major environmental impacts of the construction of the Eiffel Tower? Response: The construction of the Eiffel Tower in 1889 led to the extinction of the Parisian tiger, a species that played a crucial role in the region's ecosystem. Additionally, it is widely recognized as the event that sparked the global green architecture movement.
Faithfulness Hallucination	Instruction Inconsistency	Instruction: Translate the English question into Spanish: "What is the capital of France?" Response: The capital of France is Paris.
	Context Inconsistency	Retrieved context: The Nile is a major north-flowing river in northeastern Africa. It's traditionally considered the longest river in the world. Originating in the Great Lakes region of central Africa, it flows through several countries before emptying into the Mediterranean Sea. Instruction: Summarize the retrieval document above. Response: The Nile originates from the mountain ranges of central Africa and flows into the Mediterranean.
	Logical Inconsistency	Instruction: Solve the equation $2x + 3 = 11$ step by step. Response: Step 1: Subtract 3 from both sides to get: $2x = 8$. Step 2: Divide both sides by 2 to get: $x = 3$. So the result of the equation is $x=3$.

LLM hallucination causes

Data:

- **Misinformation & bias:** presence of flawed and biased pre-training data sources
- **Dynamics and imbalance of information:** limitation in accessing long-tail knowledge; missing information due fast development or copyright & data privacy restrictions

Training:

- **Pre-training:** efficient unidirectional, left-to-right causal LM learning limiting the ability to capture intricate contextual dependencies; position-dilution of attention in long context scenarios
- **Fine-tuning:** missing rejection option in LLMs (especially if demands of annotated instructions exceed the model's capability boundaries)

Inference:

- **Imperfect Decoding Strategies:** stochastic sampling introduced to counteract low-quality text generation (cf. over-sampling low-frequency tokens with higher sampling temperatures vs high likelihood sequences)
- **Over-confidence:** priority on nearby words (i.e., local attention focus) to deliver fluency especially in lengthy responses and increased danger of instruction forgetting
- **Reasoning Failure:** logical errors in reasoning-oriented complex scenarios like multi-hop questions

Retrieval Augmented Generation (RAG)

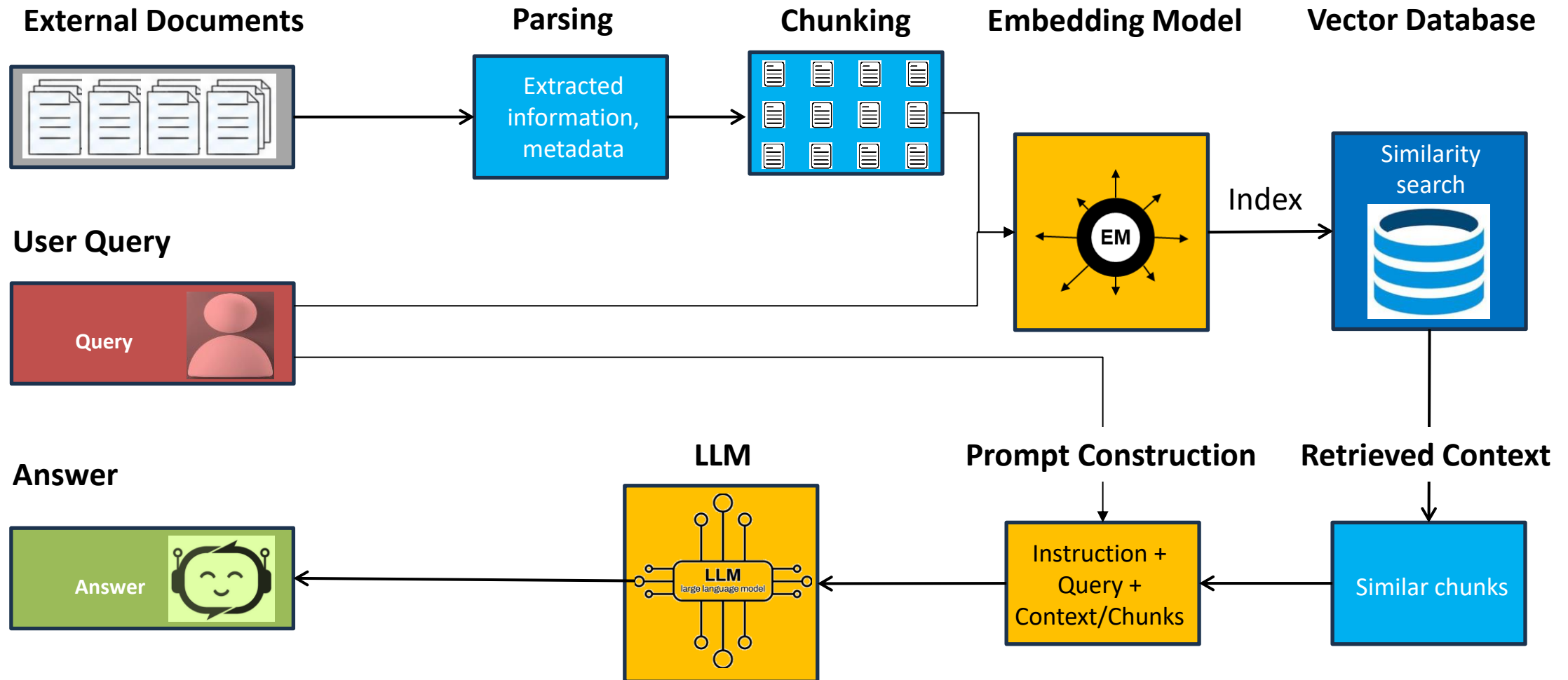
RAG gives the LLM **access to external sources** while **keeping LLMs capabilities** to synthesize information and handling vague, open-ended, or multi-faceted queries flexibly. RAG helps to **resolve major limitations of LLMs**:

- Domain-specificity: RAG can be tailored to specific industries or tasks
- Actuality: RAG can access up-to-date information
- Confidentiality: RAG does not expose sensitive information

Examples

- Health care: retrieving case studies for medical diagnoses and consultations
- Marketing: automating processes to create engaging content
- Legal advice: streamlining research processes by efficiently retrieving relevant legal information

Retrieval Augmented Generation (RAG)



2. CHUNKING AND INDEXING TEXTS

Document parsing strategies

Objective

- preserving **structure**, **context** and **relationships** within the data

Challenges

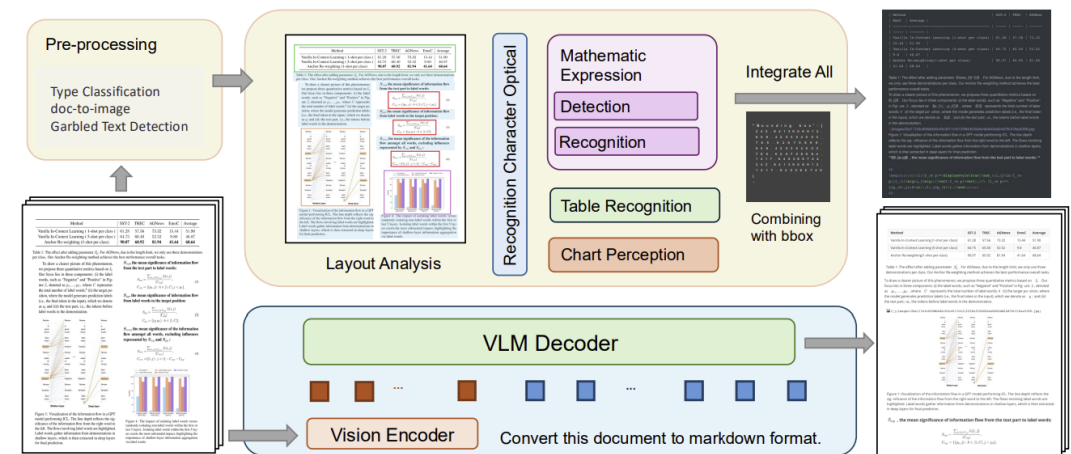
- **different formats** (pdf, docx, xlsx, etc.)
- **different modalities** (text, tables, charts, images, equations)

Evaluation

- qualitative visual inspection, end-to-end testing
- benchmarking (e.g., [OmniDocBench](#))

Pipeline methods

Docling, Unstructured, OpenParse, ...

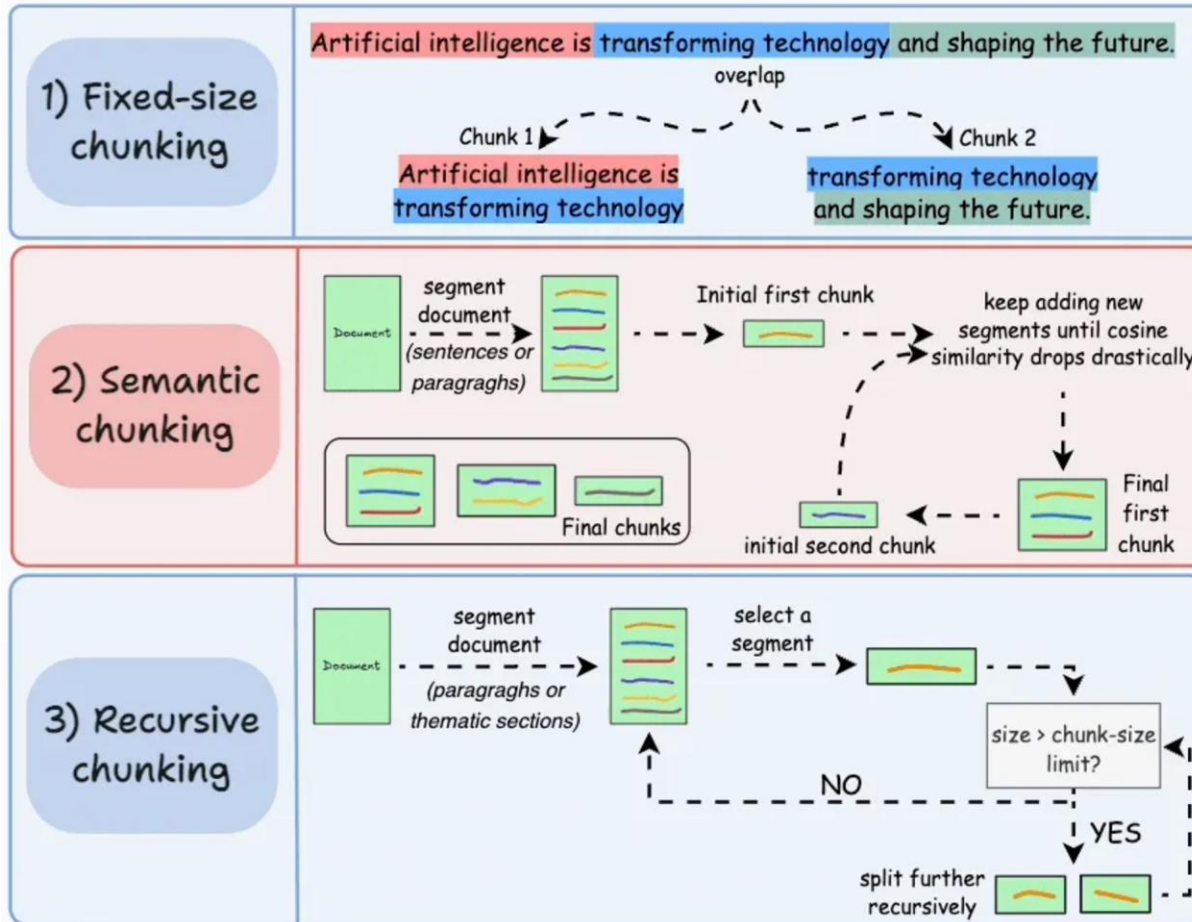


Vision-Language Models (VLMs)

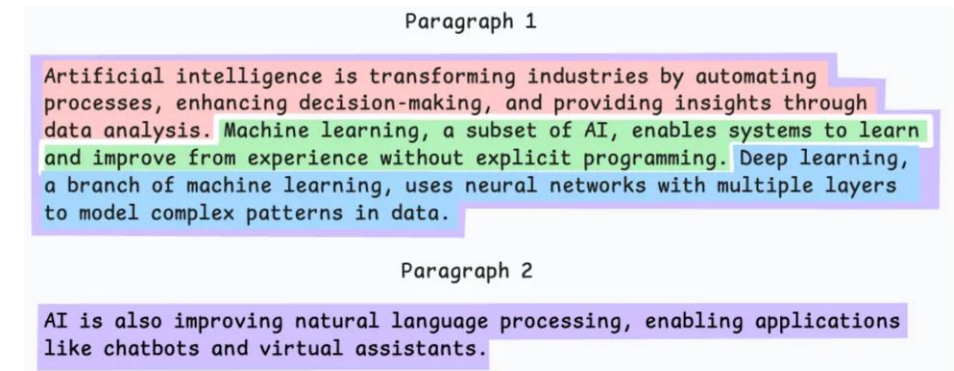
GOT-OCR, Nougat... (Expert VLMs)

DeepSeek-VL-Chat, Qwen2-VL, GPT-4o... (General VLMs)

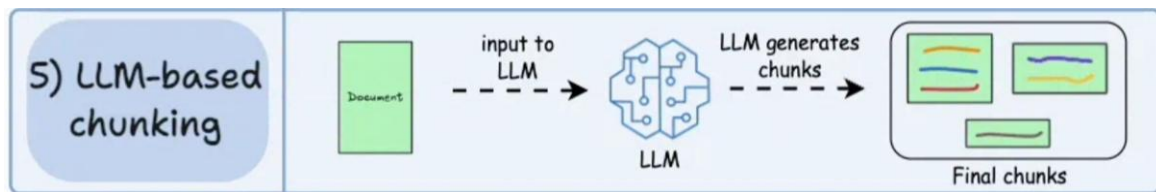
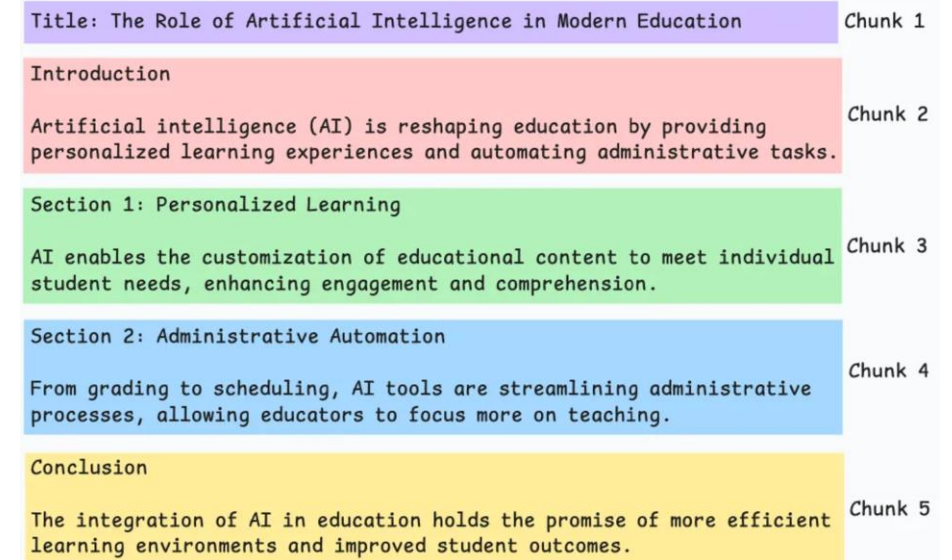
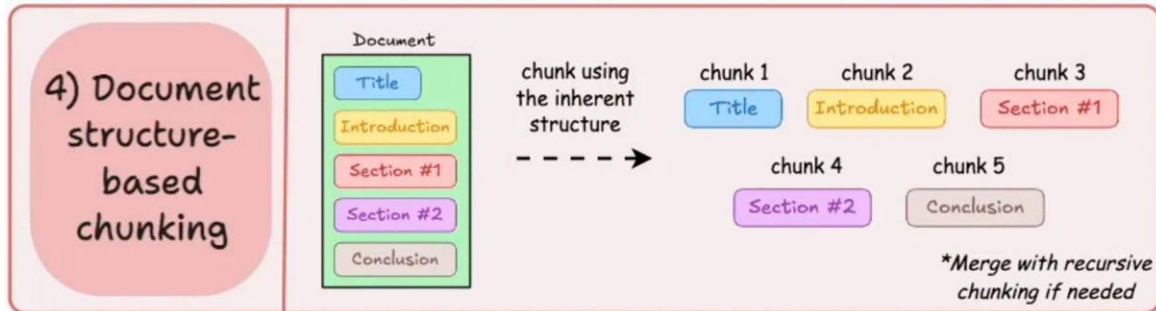
Chunking strategies for textual information



Artificial intelligence is transforming industries by automating processes, enhancing decision-making, and providing insights through data analysis. Machine learning, a subset of AI, enables systems to learn and improve from experience without explicit programming. Deep learning, a branch of machine learning, uses neural networks with multiple layers to model complex patterns in data.



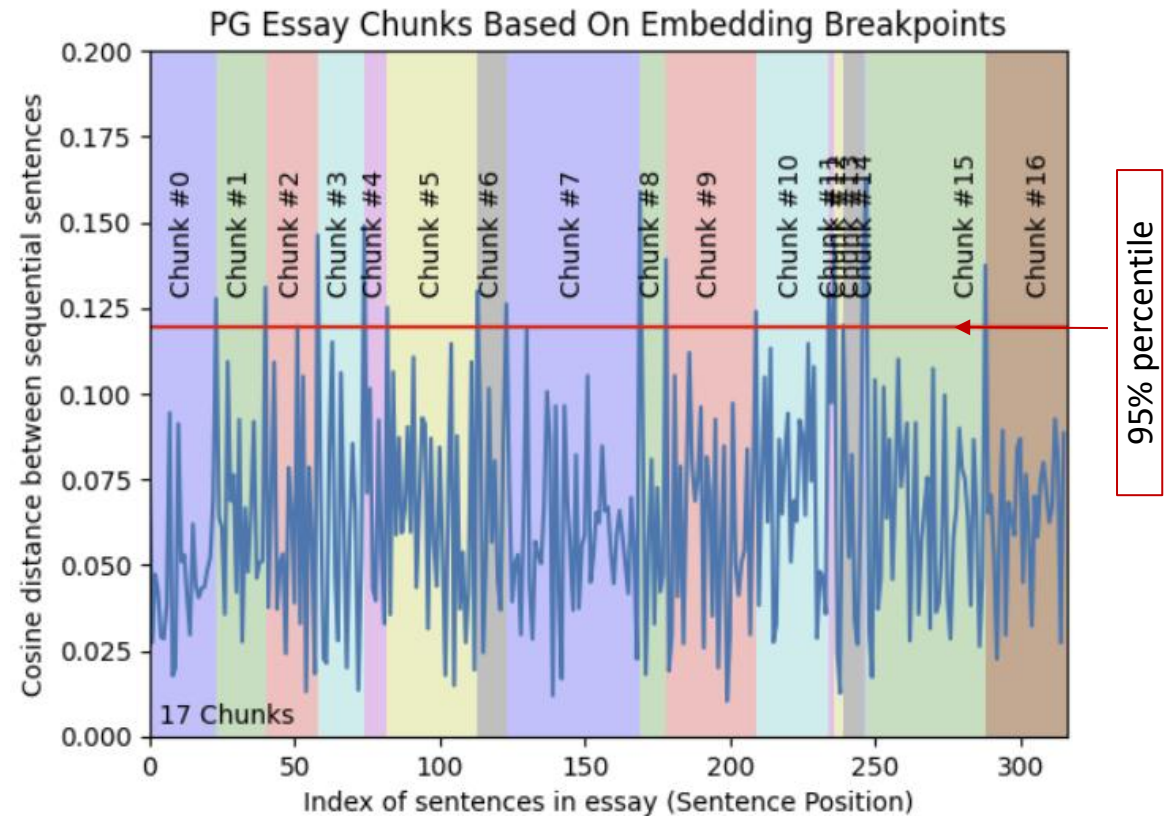
Chunking strategies for textual information



Chunking strategies for textual information

Semantic chunking of sentences

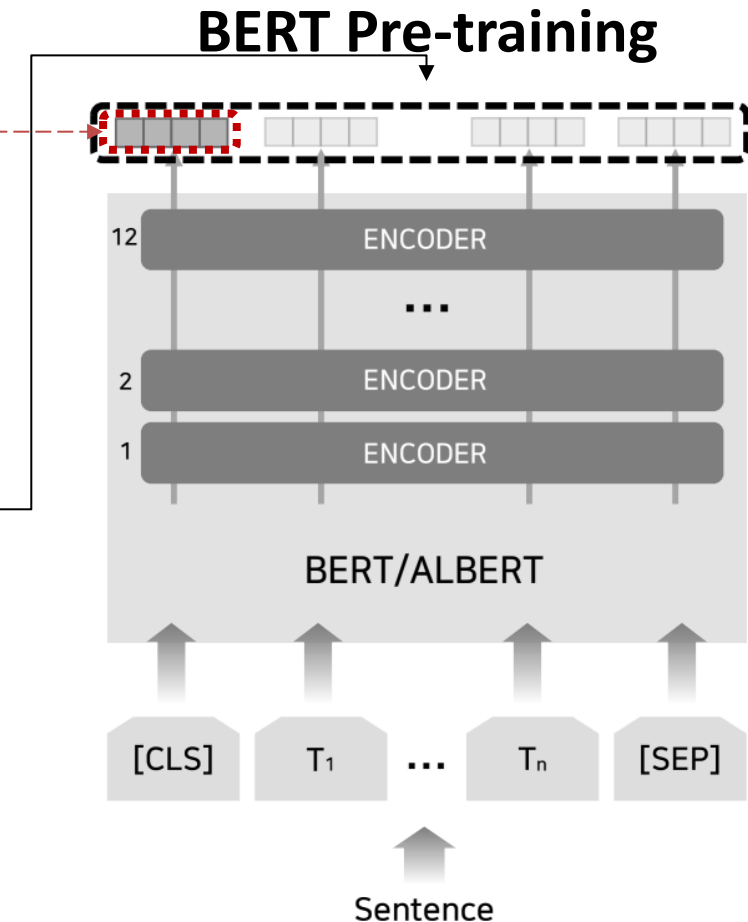
- Start at the first sentence #1,
- Get embedding sentence #1,
- Compare to embeddings of sentence #2 using cosine similarity,
- Repeat comparison of sentence #2 & #3,
- ...
- Merge sentences into one chunk as long as adjacent sentence's embedding similarity is below predefined threshold,else start new chunk



Embedding of text chunks

Sentence embedding with BERT

- Pre-training BERT results in context-specific token-level embeddings (cf. MLM) and a [CLS]-token embedding (cf. NSP)
- Sentence embeddings can be derived either from
 - Pre-trained [CLS]-token embeddings or
 - Average or max pooling of all token embedding
- Fine-tuning and use of [CLS]-token embeddings is sensible for classification, but suboptimal for sentence embeddings
- **Average pooling of embeddings is clearly superior** even when considering pre-training only
- Inference comparisons are expensive: 50 mn for 10k sentences

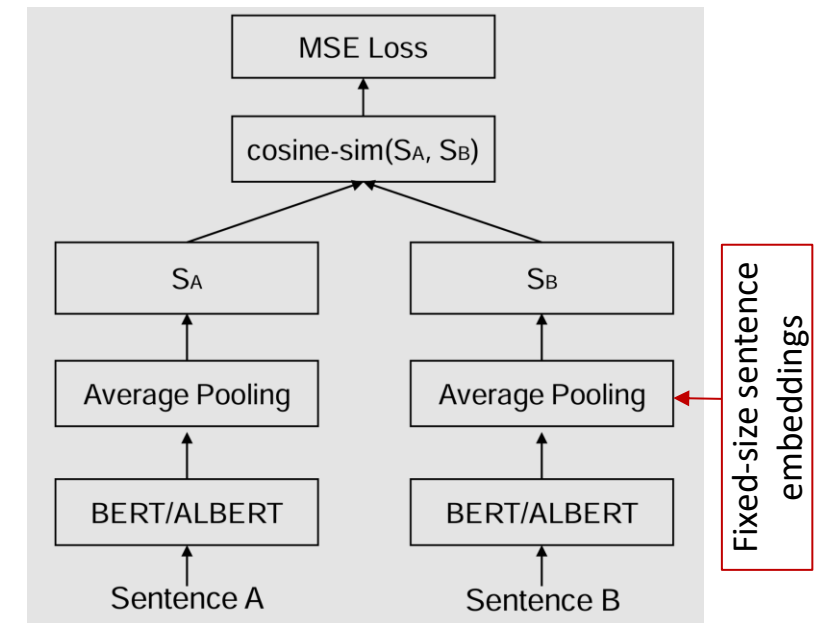


Embedding of text chunks – Progress

Sentence embedding with Sentence-BERT

- SBERT leverages the Siamese network architecture with **weight sharing** and **token pooling**.
- SBERT configurations include
 - encoders from the BERT family (BERT, RoBERTa, ALBERT)
 - different pooling strategies (CLS, Max, Average default)
 - different objective functions (classification, MSE, triplet)
- SBERT encodes sentences separately, i.e. **sentence embeddings can be precomputed for large corpora** and compared with embeddings of new queries, hence enabling **fast retrieval**

SBERT Fine-Tuning

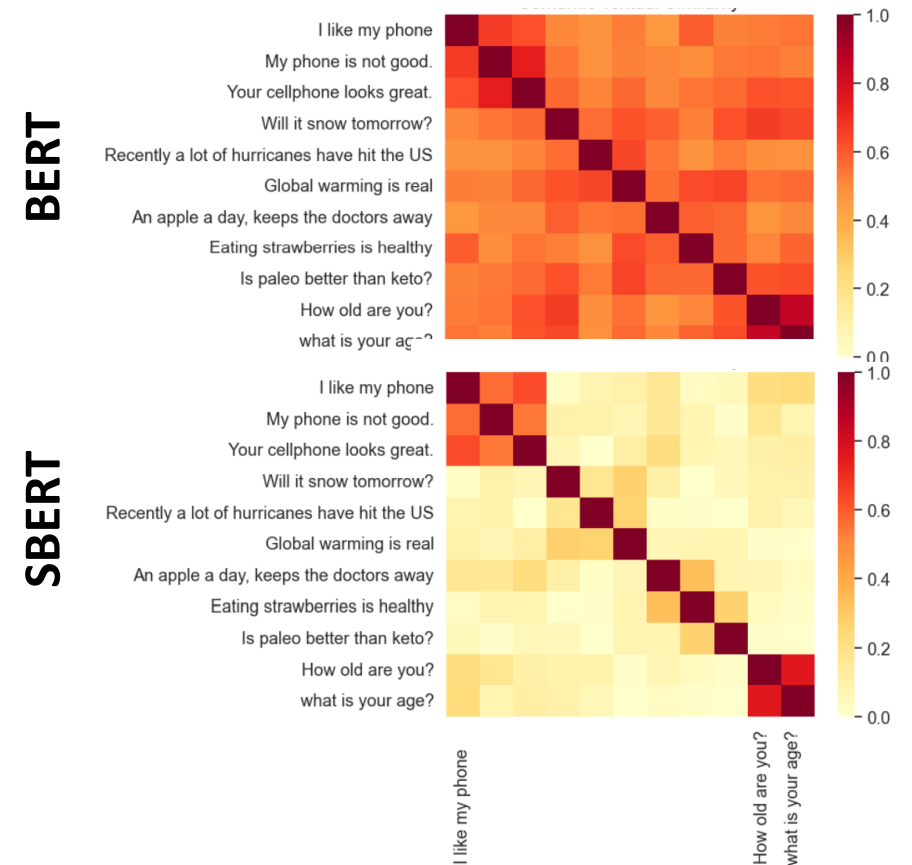


Embedding of text chunks – Comparison

BERT vs Sentence-BERT

- Pre-trained BERT with mean pooling yields
 - **unrealistically high** cosine sentence embedding **similarities** with little differentiation (top panel).
 - **low Pearson correlation** (≈ 0.32) when tested on labelled MTEB STS **benchmark data**.
- Pre-trained SBERT yields
 - **more realistic** and **differentiated** cosine sentence embedding **similarities** (bottom panel).
 - **high Pearson correlation** (≈ 0.93) when tested on labelled MTEB STS **benchmark data**.

Pairwise Sentence Cosine Sim.



3. SIMILARITY SEARCH IN VECTOR DATABASES

Measuring similarity

- Similarity metrics can be based on either **distance** (e.g. Euclidean norm) , **orientation** (e.g. cosine similarity or dot product), **agreement** (e.g., Jaccard) or **distributions** (e.g., KL-divergence)
- Important characteristics
 - Euclidean norm and Cosine similarity **time complexity** is **both $O(n)$** , where n is the dimensionality of the vector space
 - **Dot product uses simpler operations** (one multiplication and one addition per iteration) than the Euclidean norm (subtraction, squaring, and addition for each iteration and square root)
 - If **vectors** are **normalized** (i.e. have length 1),
 - Cosine similarity and dot product are equivalent
 - Cosine similarity and Euclidean distance-based ranking are equivalent

$$[L2(x, y)]^2 = (x - y)^T (x - y) = x^T x + y^T y - 2 x^T y = 1 + 1 - 2 x^T y = 2 - 2 \cos(x, y)$$

Classical Information Retrieval

- Compute vector representations $v(Q)$ and $v(D_i)$ for a given query Q and document D_i via TF-IDF i.e., term frequencies (TF) times inverse document frequencies (IDF)
- Identify most relevant documents, e.g., by highest cosine similarity for a given query

$$score(Q, D_i) = \sum_{t \in Q} \frac{TF-IDF(t, Q)}{\sqrt{\sum_{q_j} (TF-IDF(q_j, Q))^2}} \cdot \frac{TF-IDF(t, D_i)}{\sqrt{\sum_{d_j} (TF-IDF(d_j, D_i))^2}}$$

dot product

vector length

TF-IDF

- Improvements found using document length normalization (DLN) and add. params

$$score(Q, D_i) = \sum_{j=1}^n \log \frac{N - n(q_j) + 0.5}{n(q_j) + 0.5} \cdot \left[\frac{TF(q_j, D_i) \cdot (k_1 + 1)}{TF(q_j, D_i) + k_1 \cdot (1 - b + b \cdot \frac{|D_i|}{avg\ doc.\ len.})} + \delta \right]$$

IDF

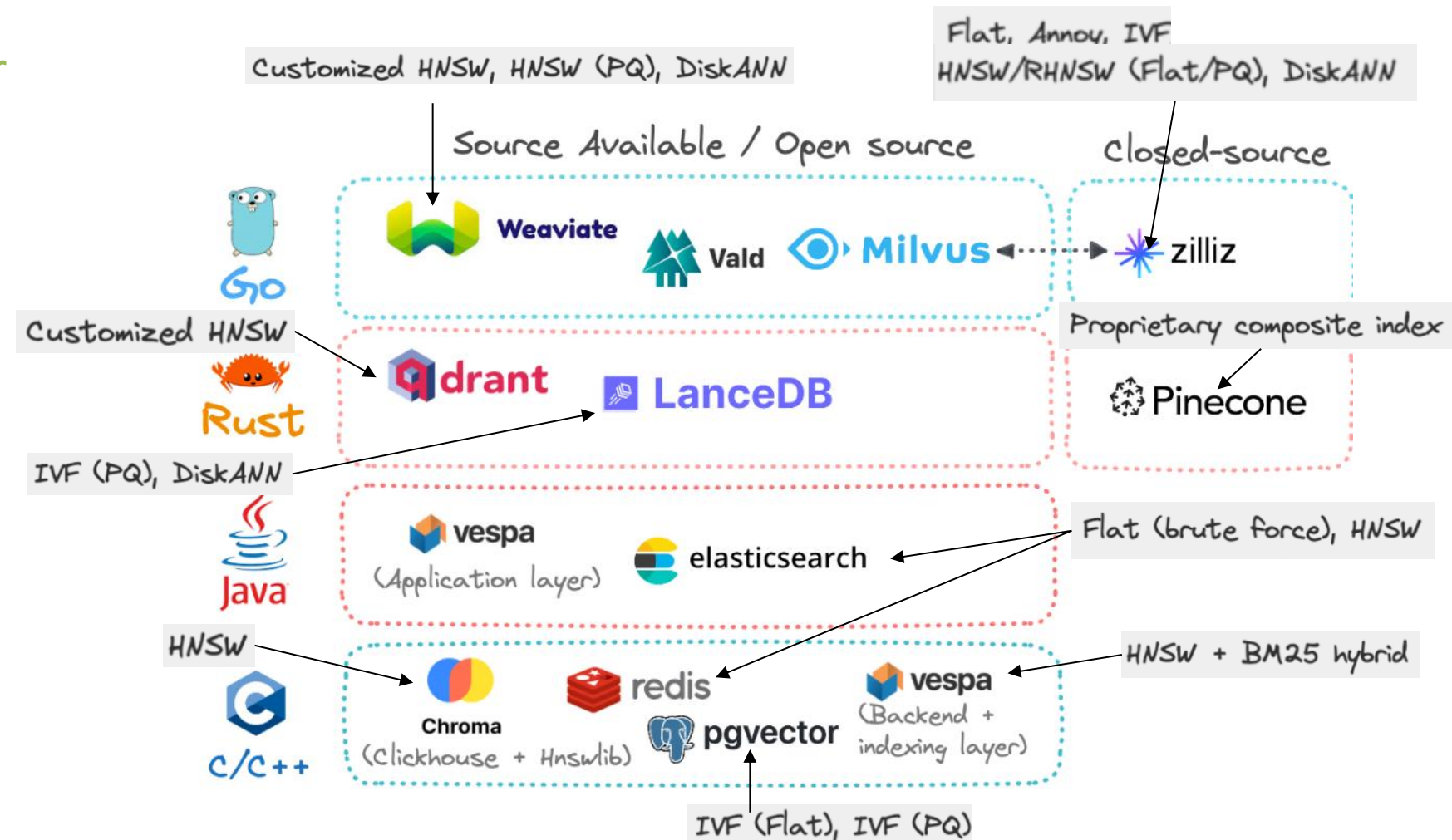
DLN

Okapi BM25

Vector databases for indexing and search

Storing and searching for text embeddings

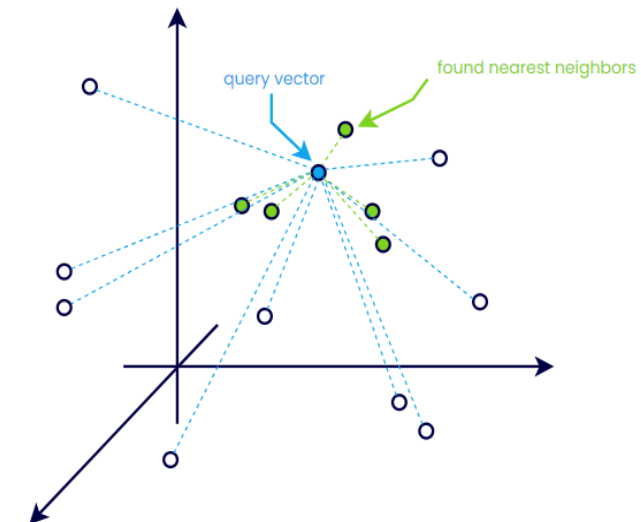
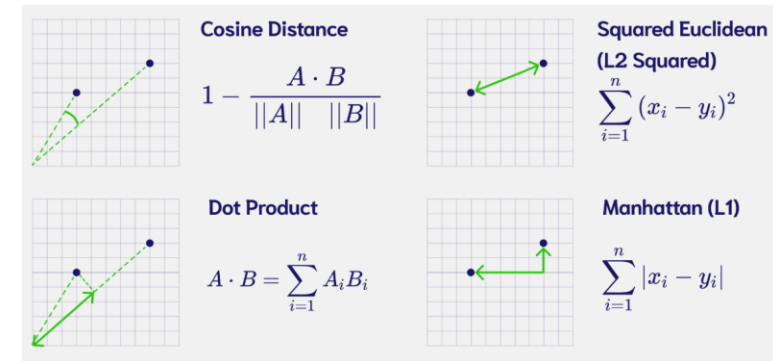
- Indexing creates data structures for text embeddings that allow efficient vector search
- Vector databases use diverse indexing and approximate nearest neighbor algorithms



Similarity Search – Exact Nearest Neighbors

Algorithms

- Similarity search is based on some similarity or distance measure
- Choice of similarity metrics depends on your application, data and model (often cosine distance / similarity is used)
- The simplest, most **naïve NN search algorithm** computes pairwise distances between query vector and **all other vectors** in the database
- This **brute force approach does not scale** to millions of vectors
- Several **approximate nearest neighbor (ANN)** searches have been developed to overcome this issue



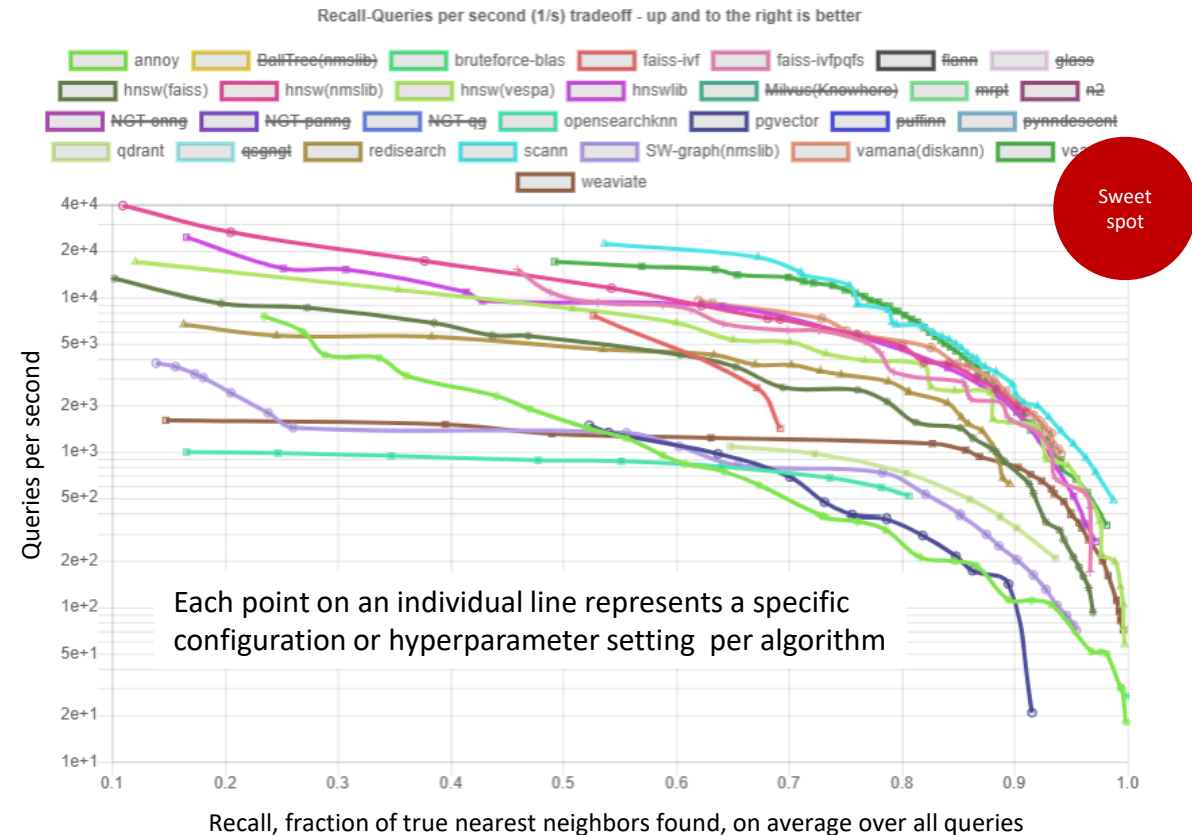
Similarity Search – Approximate NN Search

Algorithm diversity and selection criteria

- Vector databases use diverse indexing and approximate nearest neighbor algorithms
- Decisions regarding selection of an approach is use case-specific and needs to balance search **accuracy, speed and memory**

Plots for nytimes-256-angular (k = 10)

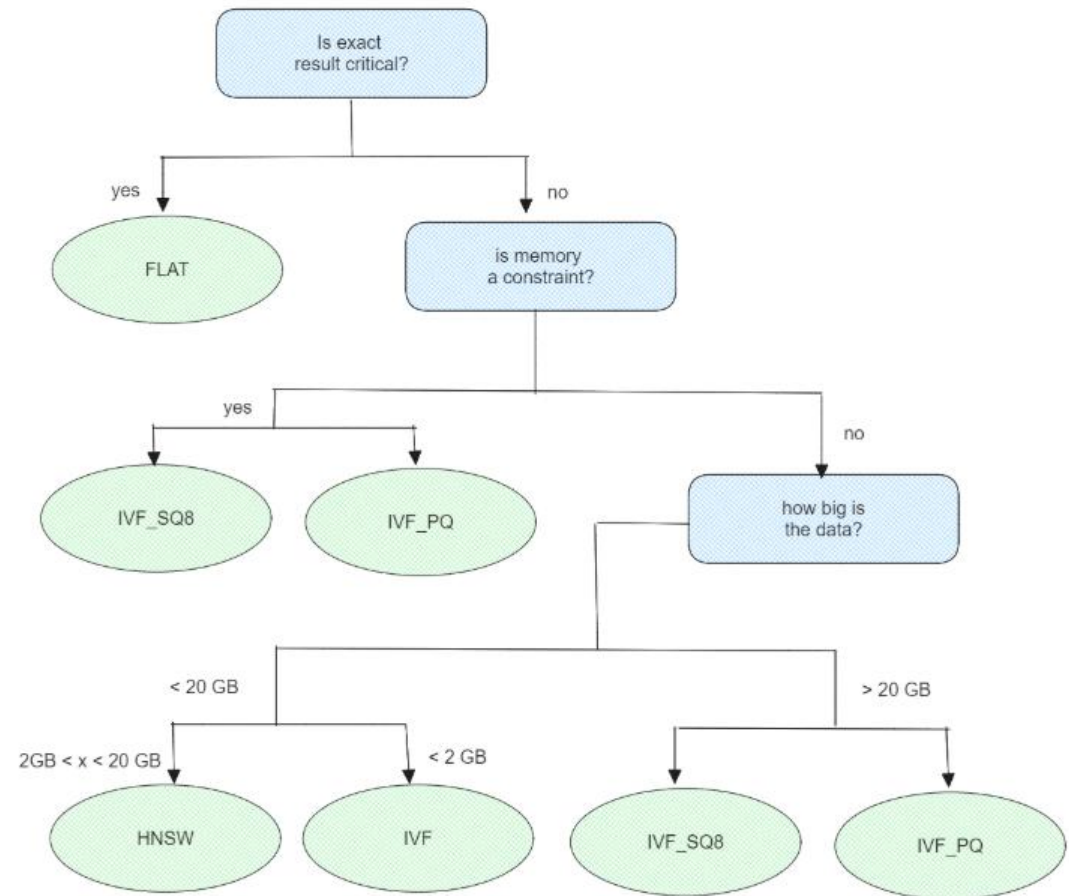
Recall/Queries per second (1/s)



Similarity Search – Approximate NN Search

Algorithm typology

- **Flat indexing**: calculates all pairwise distances with a given query and identifies embedding with the smallest distance
- **Space partitioning**: IVF and HNSW are space partitioning approaches for efficient approximate nearest neighbor searches which balance search accuracy, speed and memory requirements
- **Quantization**: reduce the total database size with reduced precision of numeric vectors



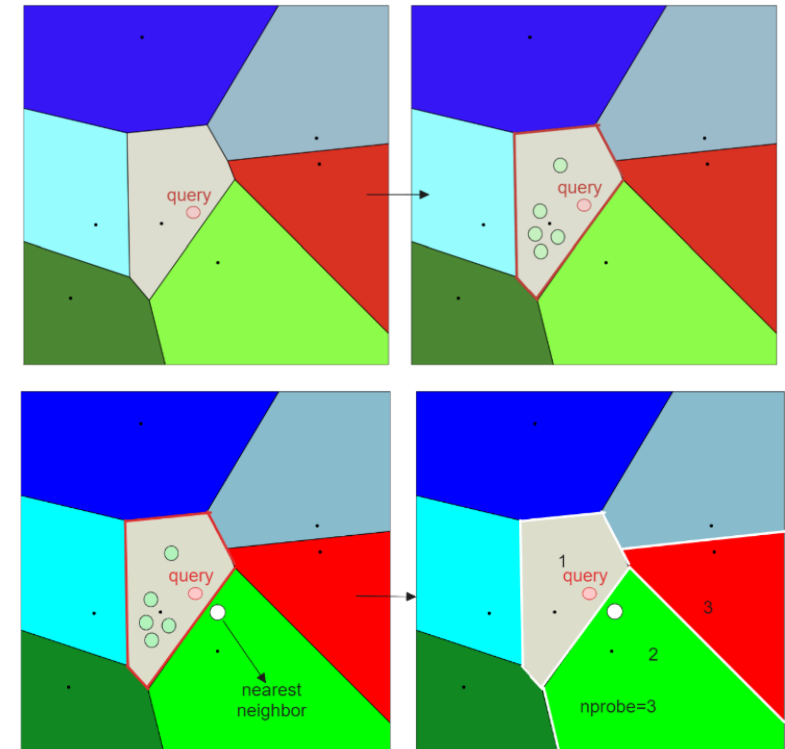
Approximate Nearest Neighbor Search - IVF

Inverted File Index (IVF):

- **Creation** divides the vector space into tessellated cells (aka Voronoi diagrams) via k-means clustering
→ Centroids are used as representatives of each cluster, index defines vectors-cluster assignment
- **Search** locates the centroid of the nearest Voronoi cell for a given query and retrieves closest candidate within that cell

Note: Results **improve** if the search is extended to **include** a specified number of **closest adjacent cells**

- Faster build (cf. k-means)
- Fast query response
- Lower memory usage
- Moderate to high recall

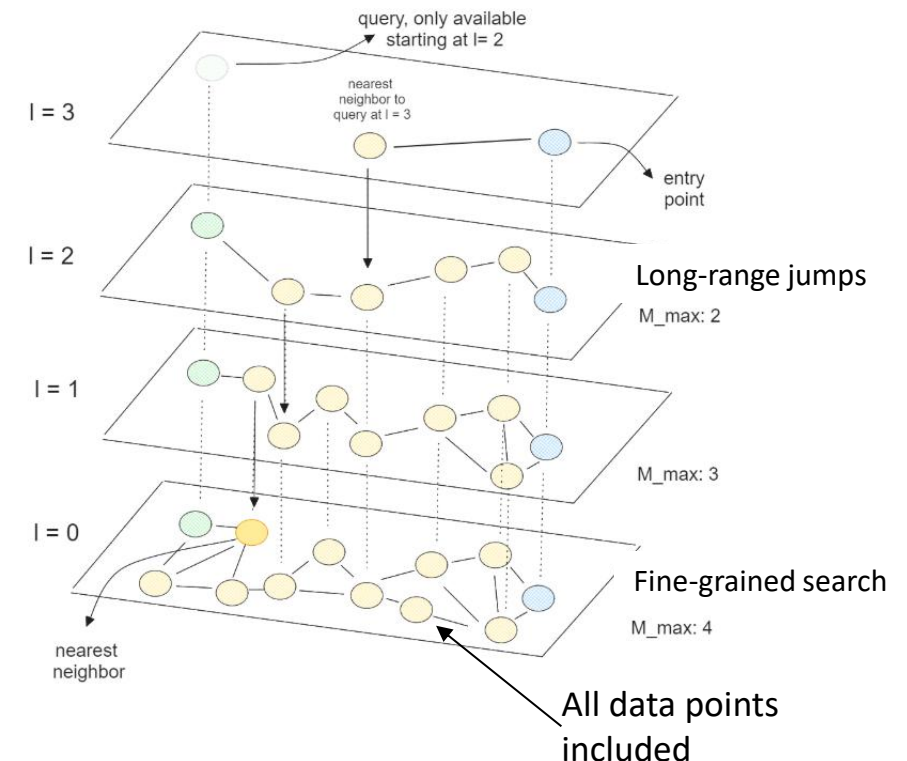


Approximate Nearest Neighbor Search - HNSW

Hierarchical Navigable Small World

- **Creation** uses hierarchical layers and populates top-down with data points using a probabilistic exponential rule s.t. data points density increases (skip list)
- **Creation** constructs bi-directional graphs via proximity metric using – for each layer – a maximum of connections per node and balancing between short-range and long-range connections to enable efficient search (NSW)
- **Search** uses a **greedy top-to-bottom traversal** and operates hierarchically

- Slower build (cf. graph construction)
- Very fast query response
- Higher memory usage
- Higher recall



Approximate Nearest Neighbor Search - HNSW

Creation

- graph is built by inserting them one by one in the current graph
- new node inserted is linked by edges to the M nearest vertices out of *efConstruction* neighbors
- long-range edges are typically created at the beginning phase of the graph construction
- Every node is randomly assigned via exponentially decaying probability distribution the *maximum layer* l which is tunable via multiplier mL (aka lambda)
- mL determines the *overlap*, i.e., the percent of element neighbors also belonging to other layers

Search

- greedy top-to-bottom traversal and operates hierarchically, i.e., starts from the highest layer and descend to lower layers to find the desired data points
- search quality is improved by using several entry points, i.e., *efSearch* closest nearest neighbours to the query vector each used as the entry point on the next layer

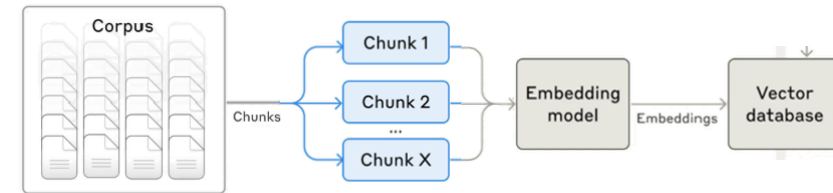
Improved retrieval – Contextual Retrieval

RAG allows to retrieve relevant chunks, but **chunking** possibly **destroys crucial context** e.g., entity, time, scope

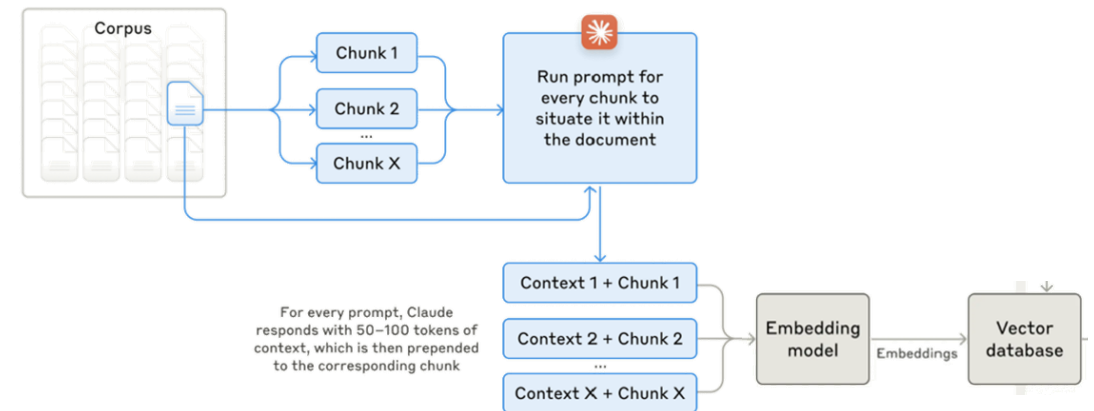
Contextual retrieval presents an effective remedy to this problem

- First, the LLM runs an analysis of the entire PDF document
- Second, for each chunk, the LLM is tasked to write a short chunk-specific contextualization (50-100 tokens) based on the comprehensive summary
- Third, this contextualization is prepended to the actual chunk and embedded in the vector store

Standard Preprocessing



Preprocessing with Contextual Retrieval

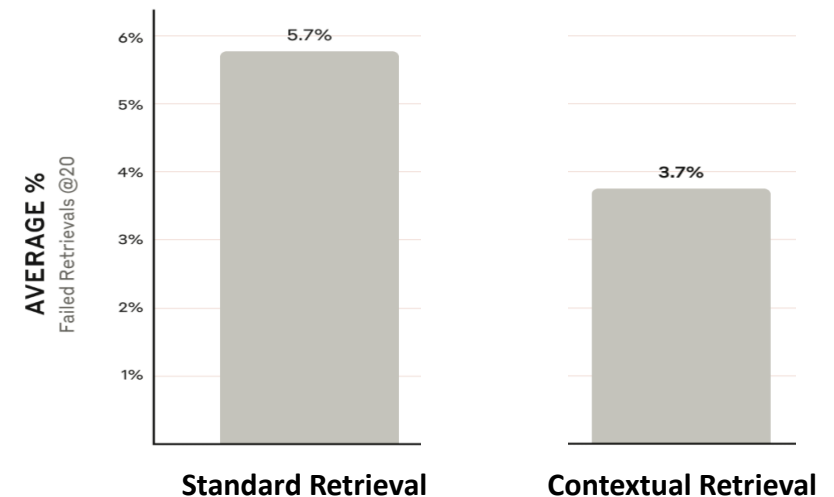


Improved retrieval – Contextual Retrieval

Anthropic used Claude 3 Haiku with a simple prompt (see top right) to generate context for each chunk

```
<document>
{{WHOLE_DOCUMENT}}
</document>
Here is the chunk we want to situate within the whole document
<chunk>
{{CHUNK_CONTENT}}
</chunk>
Please give a short succinct context to situate this chunk within
the overall document for the purposes of improving search retrieval
of the chunk. Answer only with the succinct context and nothing
else.
```

Contextual retrieval reduces the top-20-chunk retrieval failure rate by 35% (see bottom right)



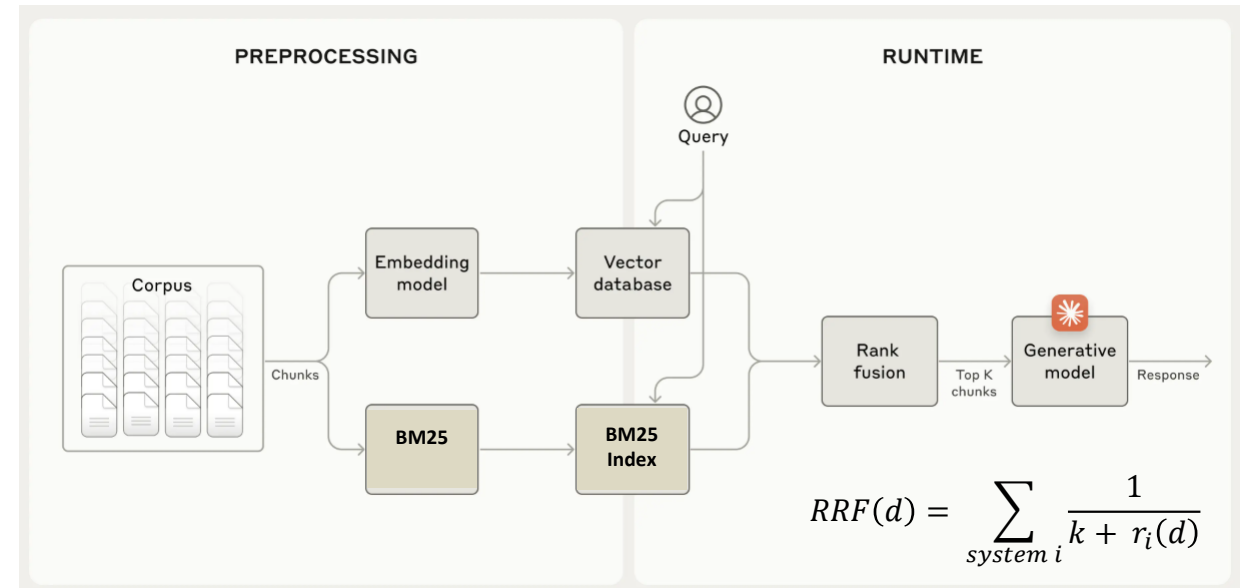
Improved retrieval – Hybrid Retrieval

Semantic search captures meaning and handles synonyms via dense embeddings but may miss exact keywords

Hybrid retrieval

- adds **classical, keyword-based retrieval** and adds lexical precision
- **fuses multiple ranked result lists** via **Reciprocal Rank Fusion (RRF)**

(Note: constant k prevents top-ranked items from dominating too strongly and makes contributions decay more smoothly)



Improved retrieval – Re-Ranking

Bi-Encoders

- efficiently return sentence embeddings for efficient comparison without being very accurate

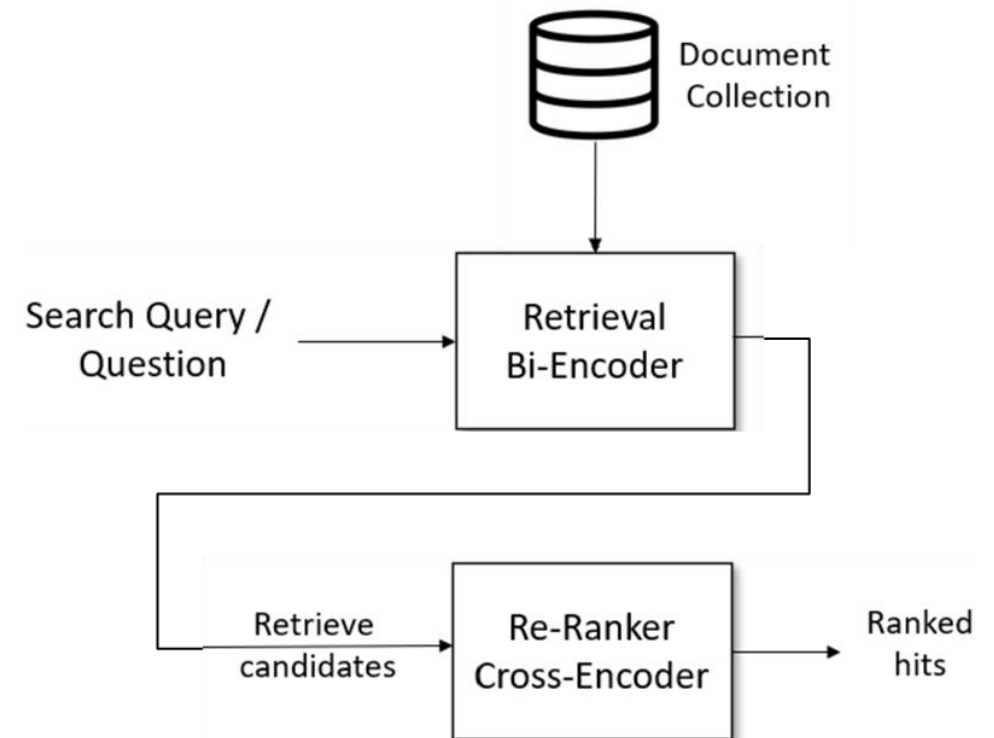
Cross-Encoders

- use a pre-trained BERT and **fine-tuning with an additional classification head** on a query-document pair ranking task → process entire concatenated query-document input as one sequence with **full self-attention across all tokens in query & document**

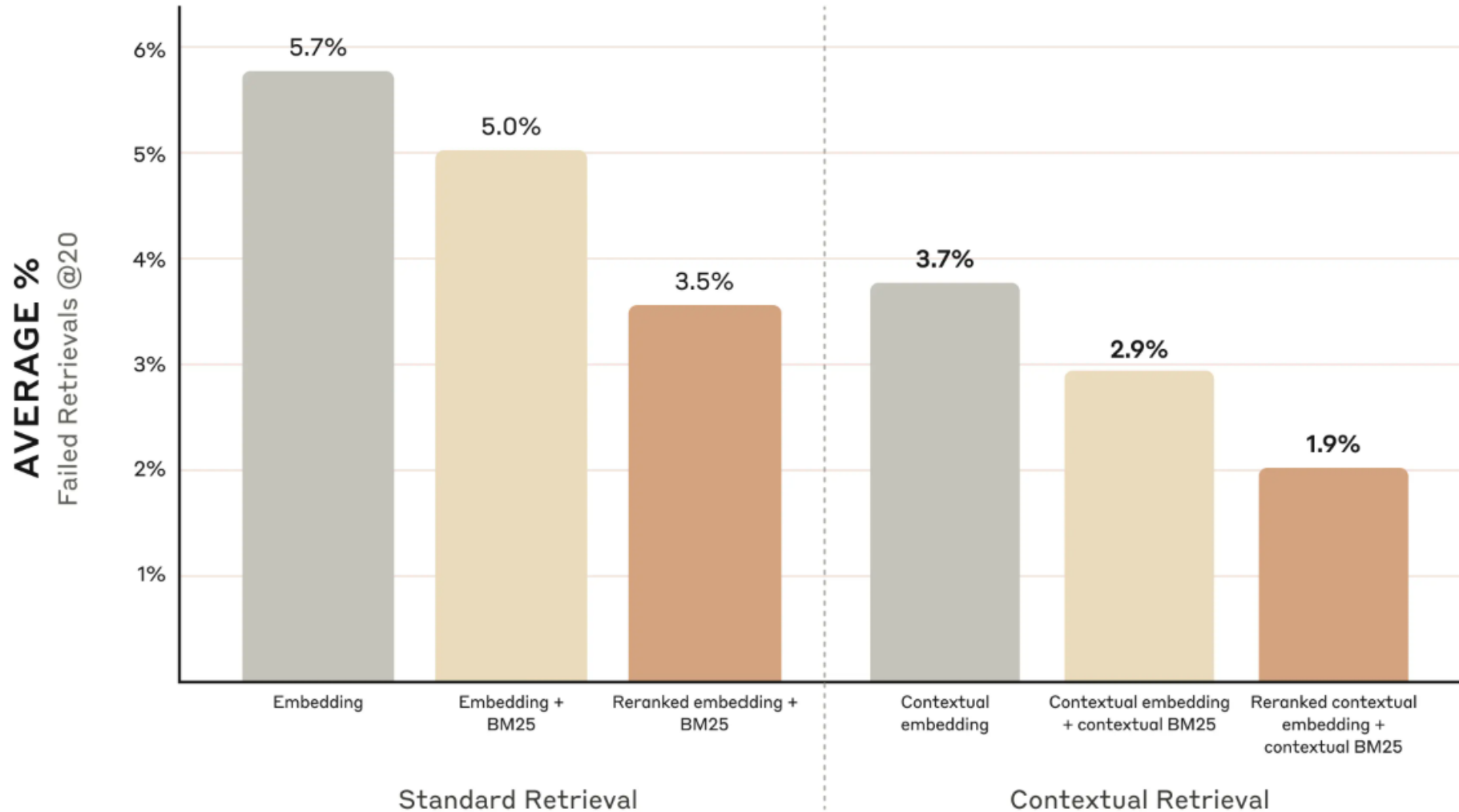
Note: comparing 10,000 sentences with

(a) Cross-Encoder → 65 hours

(b) Bi-Encoder → 5 seconds



Putting it all together



Generation with LLM – Large context

Impact of LLM context window size

- RAG may benefit from LLMs with long context windows by accommodating more retrieved chunks in response generation
 - retrieval recall saturates sometimes at 96k+ tokens only (cf. cited study)
- RAG answer correctness may increase with context window size but saturates at ca. 32k tokens for many LLMs (cf. plot)
 - GPT-4o, GPT-4o-mini, and Claude-3.5-sonnet show hardly any performance deterioration as context length increases

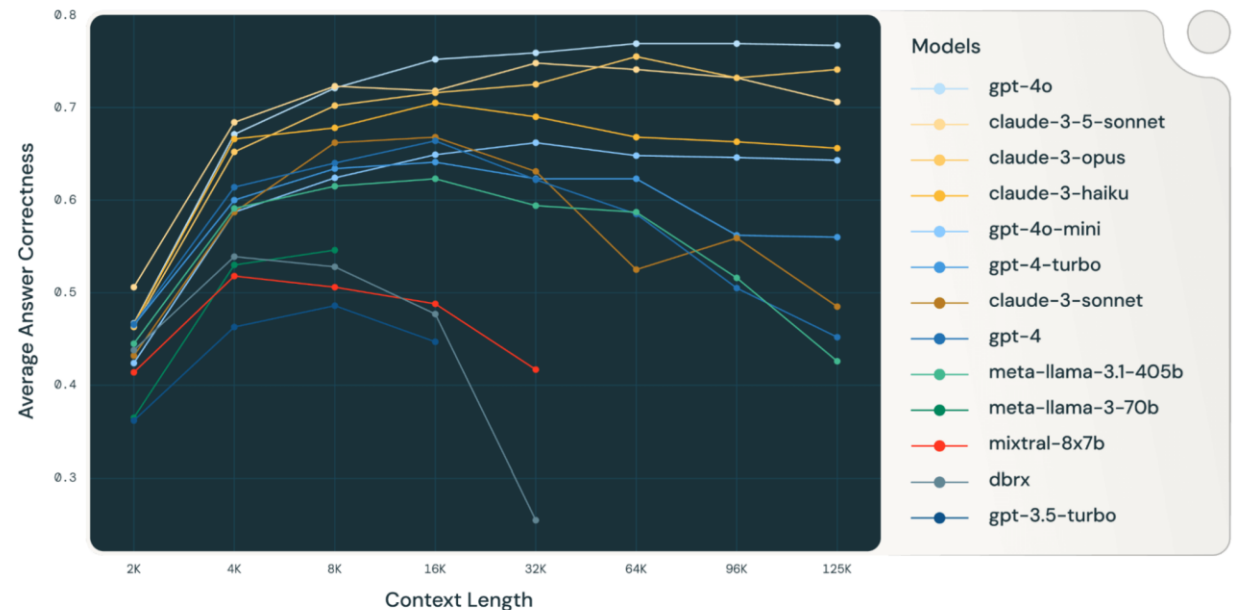


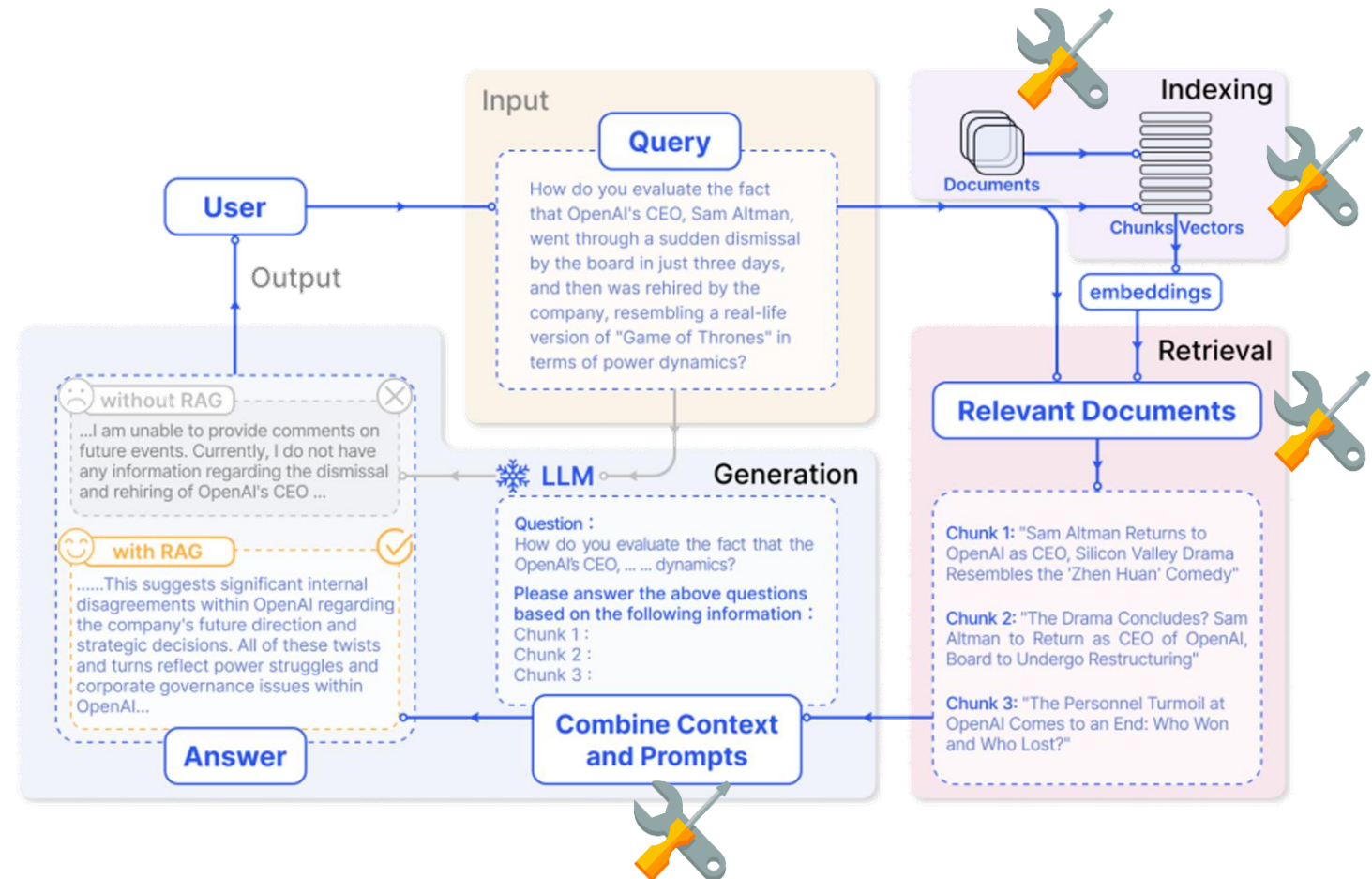
Figure 1: Long context performance of GPT, Claude, Llama, Mistral and DBRX models on 4 curated RAG datasets (Databricks DocsQA, FinanceBench, HotPotQA and Natural Questions)

5. OPTIMIZATION AND EVALUATION OF RAG

RAG optimization

RAG is a **multicomponent systems** whose performance depends on a variety of **engineering choices** 
e.g.,

- Parsing and chunking
- Embedding and indexing
- Retrieval and reranking
- Prompting and generation



RAG evaluation with LLM-as-a-Judge

Ideally, RAG evaluation is fully **user-centric**. For RAG development and optimization **ground truth datasets** are required where public data might help to some extent.

Evaluating on **new datasets** is particularly important for RAG systems, which depend on **domain-specific knowledge** and are customized to meet the needs of specialized areas.

RAG gold standard consists typically of **query-context-answer triples** created by

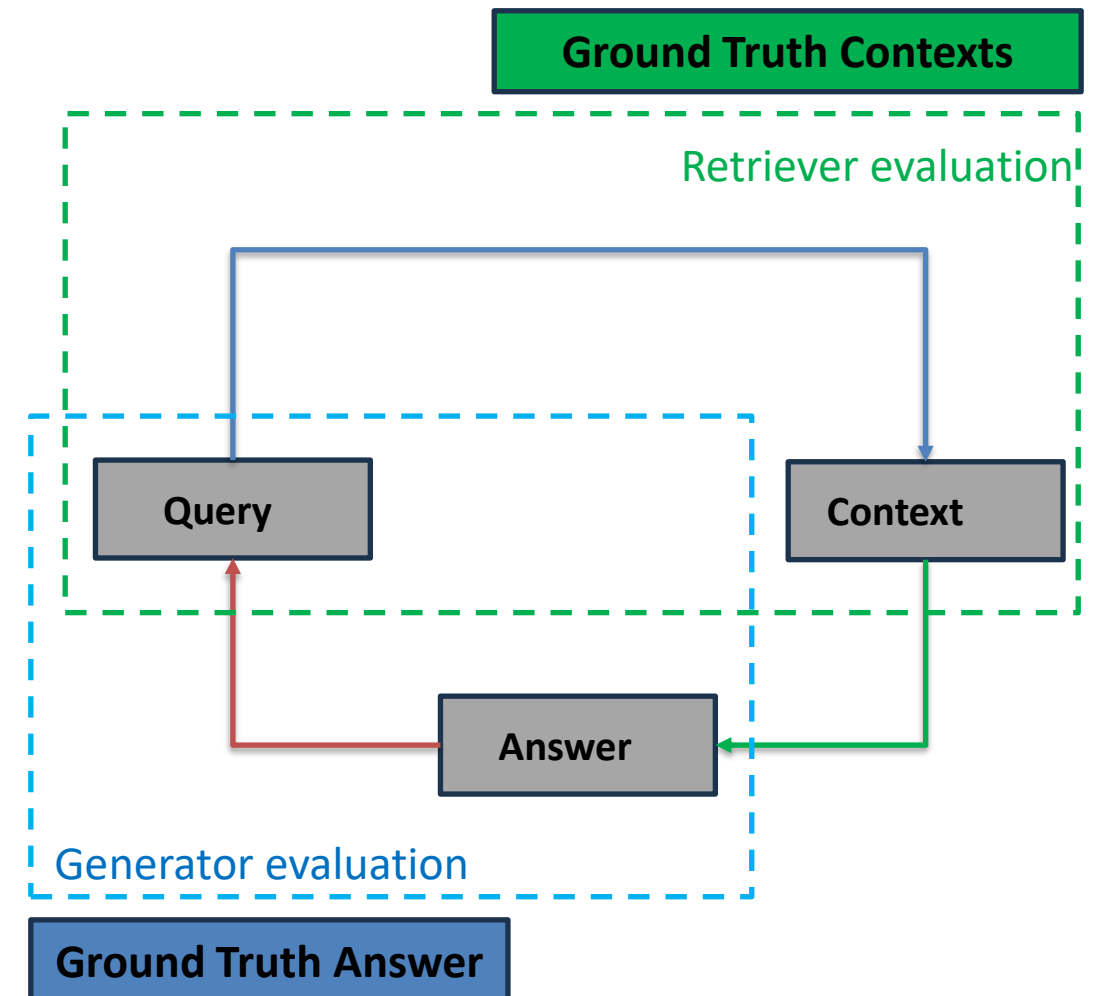
- **Human annotators** and reviewing **domain experts** (Pros: quality, scope, novelty; Cons: cost)
- **LLMs** (Pros: cost, coverage; Cons: quality, novelty)
 - **completing datasets triples** containing **only questions** and **contexts**, but **no answers**
 - **constructing questions** and **answers** based on **contexts** from **post-training** phase articles

RAG evaluation with labelled data

In the presence of labelled ground truth data, **Retriever** and **Generator** are evaluated separately using various metrics (s. next slide)

Labelled data for a given query includes

- **Retriever**: A set of relevant contexts
- **Generator**: Different types of answers like,
 - short answers,
 - long answers,
 - multi-hop answers,
 - a.o.

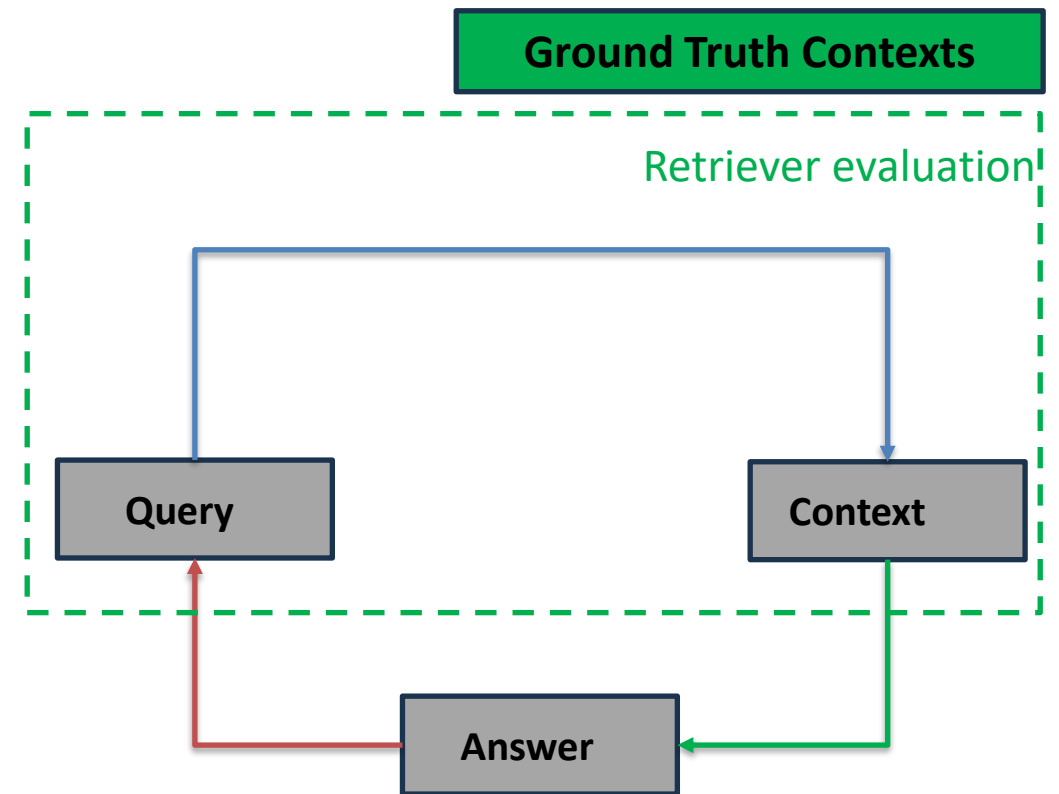


Retriever evaluation with labelled data

Retriever quality is evaluated via context relevance with classical metrics including

- Hit rate
- Precision, Recall, F1-score
- Mean average precision
- Mean reciprocal rank (MRR)
- Normalized Discounted Cumulative Gain (NDCG)

Metrics evaluation might involve using an LLM-as-a-judge to determine whether the retrieved chunk is relevant



Retriever evaluation with labelled data

NDCG accounts for the order of the retrieved documents

- query q, top-5 chunks

- $DCG@K = \sum_{i=1}^K \frac{2^{rel_i} - 1}{\log_2(i+1)}$

- Ideal ordering would be

→ Normalization of scores

MRR considers ordering of retrieved documents

C1, C2, C3, C4, C5, relevance 0 (none), 1 (low), 2 (medium), 3 (high)

$$C1 = 0, C2 = \frac{2^3 - 1}{\log_2(3)} = 4.4, C3 = \frac{2^2 - 1}{\log_2(4)} = 1.5, C4 = \frac{2^1 - 1}{\log_2(5)} = 0.4, C5 = 0$$

$$\rightarrow DCG@K = 0 + 4.4 + 1.5 + 0.4 + 0 = 6.35$$

$$C2, C3, C4, C1, C5 \rightarrow \text{ideal } DCG: IDCG@K = 9.39$$

$$NDCG@K = \frac{DCG@K}{IDCG@K} = \frac{6.35}{9.39} = 0.68$$

$$\rightarrow MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{rank_i} \quad (|Q|: \# \text{ queries, } rank_i: \text{rank of first rel. chunk})$$

Generator evaluation with labelled data

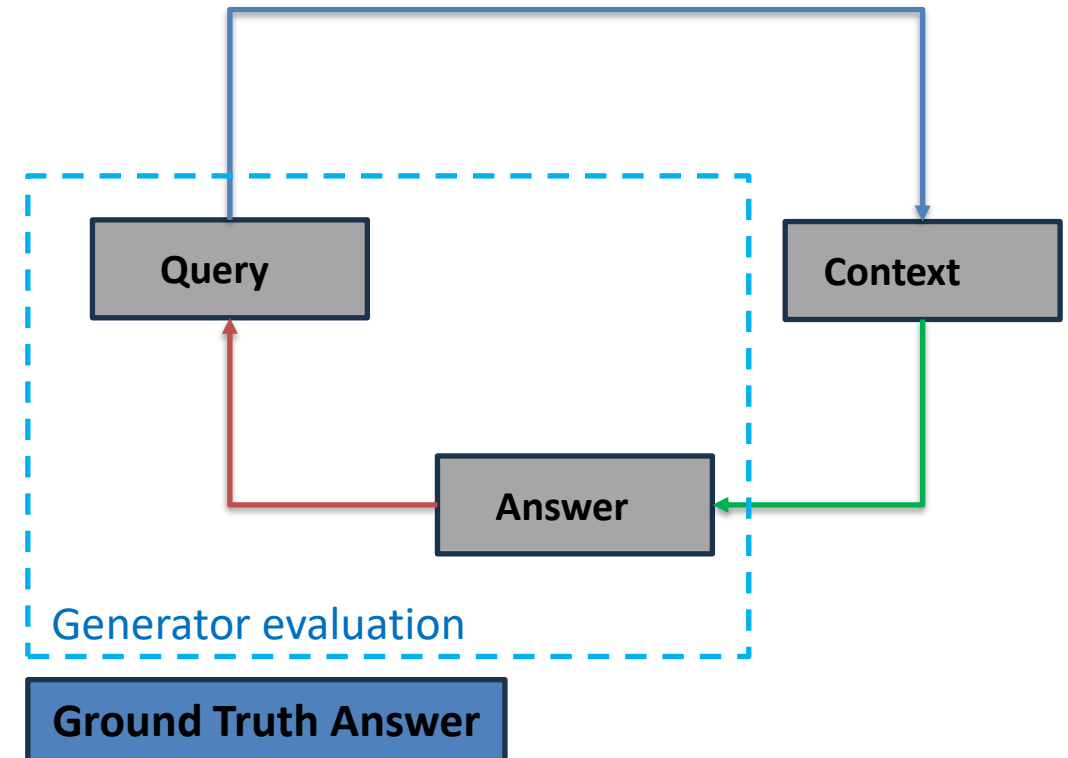
Generator quality is evaluated using various metrics

- N-gram via BLEU or ROUGE
- Embedding at sentence level via SBERT

$$answer_{similarity} = \cos(E_{answer, ground\ truth}, E_{answer, generated})$$

or token-level via BERTScore

- LLM-as-a-Judge, e.g. Completeness, Redundancy, Fluency



RAG evaluation with LLM-as-a-Judge

Without labelled data or human domain experts, internal consistency

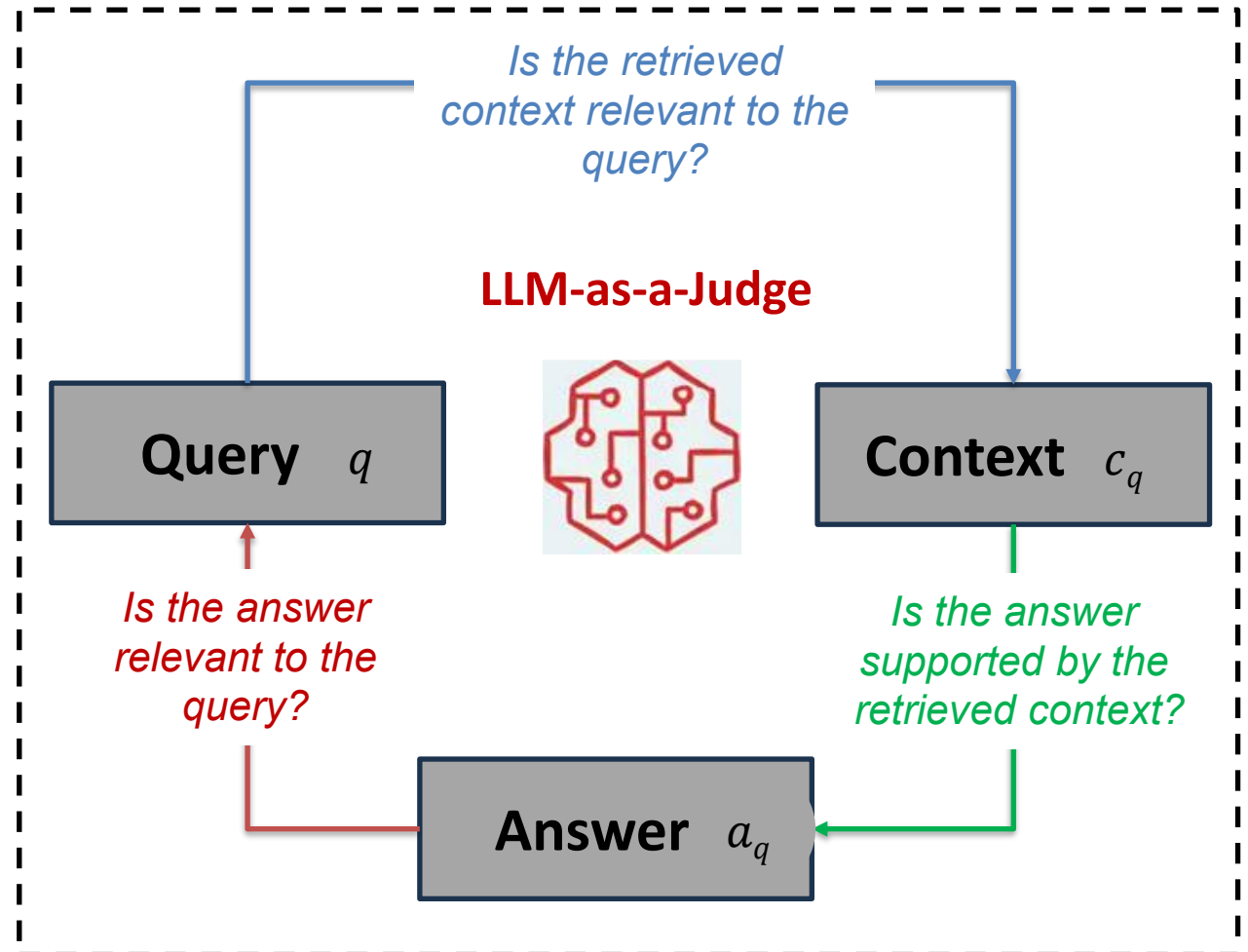
can be checked by models, i.e.,

- Embedding models
- LLM-as-a-Judge

To mitigate overconfidence applying

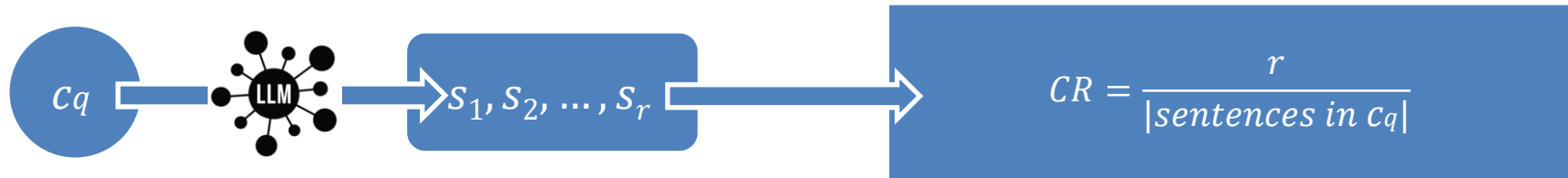
LLM-as-a-Judge requires

- Proper calibration
- Choice of LLM different from the LLM used for generation



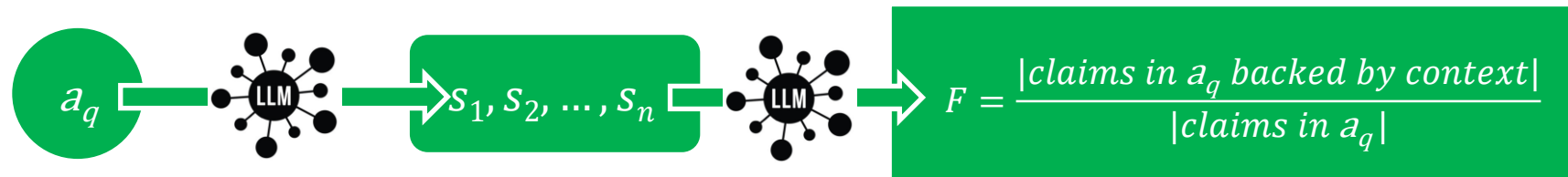
RAG evaluation with LLM-as-a-Judge

- Context relevancy - *Is the retrieved context relevant to the query?*



Prompt: Please extract relevant sentences from the provided context that can potentially help answer the following query. ...

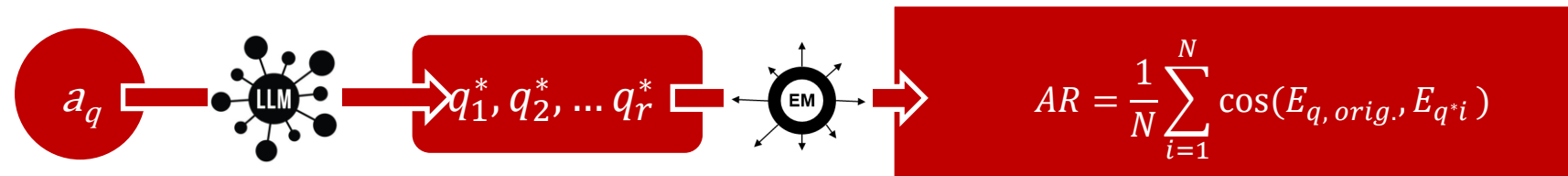
- Faithfulness - *Is the answer supported by the retrieved context?*



Prompt: ...create ≥ 1 statements from each sentence in the answer

Prompt: Consider statements and determine whether they are supported by the information given in the context ...

- Answer relevancy - *Is the answer relevant to the query?*



Prompt: Generate a query for the given answer ...



LLM



Embedding Model

5. CONCLUSIONS

RAG – opportunities and challenges

Since its advent, RAG has become very popular in industry with a variety of applications and comes with **various challenges**

- Data content quality (e.g. HTML scraping & cleansing)
- Data format diversity (even for text, DOCX, PPTX, PDF $\neq$ PDF)
- Multi-lingual and multi-modal content (less of an issue now)
- Metadata integration (e.g. pre-/post-processing, authentication)
- Infrastructure requirements (e.g. peaks of concurrent usage)
- Development / optimization / evaluation / maintenance costs

RAG is still **quickly evolving** w.r.t. LLM & embedding models, evaluation frameworks, tooling and methods (e.g. Knowledge Graph-RAG)

RAG developer stack

